Roteiro: vídeo de apresentação (Tech Challenge Fase 2)

Gabriel Wesley (RM376899) · Gustavo Leite (RM375733) · Luiz Fellipe (RM376854)

Duração: até 15 minutos. Os tempos entre colchetes são sugestões; ajustem no ensaio.

A ordem segue o padrão da Fase 1:

1. **Notebook** `notebooks/02_demo_fase2.ipynb`, célula por célula, mostrando cada chamada e cada retorno (≈ 10 min).
2. **Aplicação web**, com tudo integrado (≈ 4 min).

Os números das seções abaixo (1.1, 2.4, 4.8...) são os títulos das células do notebook.

Divisão sugerida:

- **Gabriel:** abertura, seção 1 (dados), seções 5 e 6 (API, monitoramento, testes), app (avaliação e relatório) e encerramento.
- **Gustavo:** seções 2 e 3 (algoritmo genético e experimentos) e app (AG ao vivo).
- **Luiz:** seção 4 (LLM) e app (paciente e inspetor).

Antes de gravar

- ☐ `.env`**:** precisa estar com `GROQ_API_KEY` (ou `LLM_API_KEY`). A célula 0.1 tem de imprimir `Provedor da LLM neste notebook: groq`.
- ☐ **Cota do Groq:** o notebook faz cerca de 7 chamadas à LLM e o app mais umas 5. O plano gratuito permite 1.000 por dia, mas tem limite por minuto. Se aparecer `HTTP 429; nova tentativa...` no notebook, é o tratamento de limite de taxa funcionando: esperem e comentem, se quiserem.
- ☐ **Kernel:** `Restart Kernel` e rodar **uma célula de cada vez** enquanto falam. Não usar "Run All".
- ☐ **Servidores:** deixar a API (`uvicorn src.api.main:app --port 8000`) e o front (`cd frontend && npm start`) já rodando em outro terminal, para a parte final.
- ☐ **Ensaio:** fazer o ensaio do app com o provedor **offline** e trocar para **groq** só na gravação.
- ☐ **Tela:** VS Code em tela cheia, com fonte grande.
- ☐ **Textos da LLM:** mudam a cada execução. Não leiam um texto decorado; comentem o que aparecer.

Abertura (Gabriel) [40s]

> *"Somos Gabriel, Gustavo e Luiz, e este é o Tech Challenge da Fase 2, Projeto 1. Na Fase 1 treinamos dois modelos para apoiar a triagem de câncer de mama: uma Regressão Logística e um Random Forest, com hiperparâmetros escolhidos à mão. Agora fazemos duas coisas: um* ***algoritmo genético, implementado do zero****, otimiza esses hiperparâmetros, e uma* ***LLM*** *explica cada resultado em linguagem natural para o médico. Vamos mostrar o notebook passo a passo e, no final, a aplicação web com tudo integrado."*

Célula 0.1 (ambiente): "Carregamos o `.env` com a chave do Groq. Sem chave, o notebook usa um provedor offline, que é um template determinístico."

1. Dados e ponto de partida (Gabriel) [1 min]

1.1: "Mesmo dataset e **mesmo split** da Fase 1: 455 pacientes de treino e 114 de teste, 42 delas malignas. São exatamente as mesmas da Fase 1, o que deixa a comparação justa. E o mais importante: **o algoritmo genético nunca vê o teste**."

1.2 (duas células): "Estes são os hiperparâmetros fixos da Fase 1, com limiar de 0,5, e o desempenho deles no teste. A Regressão Logística deixava passar 2 casos malignos, e o Random Forest, 3. É esse número que queremos reduzir."

2. Algoritmo genético (Gustavo) [2min45]

2.1 (tabela de genes): "Cada indivíduo é um conjunto de hiperparâmetros. O cromossomo é um vetor entre 0 e 1, e cada gene é decodificado conforme o tipo: real, com escala logarítmica para o `C`, inteiro ou categórico. Um detalhe importante: **o limiar de decisão também é um gene**. É ele que troca falsos negativos por falsos positivos."

2.1 (decodificação): "Aqui um cromossomo aleatório vira um modelo concreto."

2.2 (operadores): "Implementamos três seleções (torneio, roleta e ranking), três cruzamentos (uniforme, um ponto e BLX-α) e duas mutações (gaussiana e uniforme), além de elitismo. Na tabela, o BLX-α sorteia o filho num intervalo em torno dos pais, e a mutação desloca alguns genes. No torneio, os indivíduos de maior fitness são escolhidos mais vezes."

2.3 (fitness): "A fitness é uma validação cruzada de 5 folds, só no treino: 45% recall, 35% F1 e 20% ROC-AUC. O recall pesa mais porque não queremos deixar passar câncer. O F1 impede a solução trivial de chamar tudo de maligno. O modelo original tem fitness 0,967."

2.4 (AG ao vivo, as duas células): "Rodamos o AG agora, com população 10 e 6 gerações, em segundos. A tabela mostra cada geração: o melhor indivíduo, a média da população subindo e a diversidade caindo, que é a população convergindo. A coluna de avaliações cresce menos que população × gerações porque temos cache: um indivíduo repetido não é treinado de novo. No gráfico, o AG já começa acima do modelo original."

3. Experimentos e comparação (Gustavo) [2 min]

3.1: "Comparamos 5 configurações do AG, definidas no `configs/experiments.yaml`: população pequena, BLX com torneio, roleta com um ponto, mutação alta e uma população semeada com o modelo original."

3.2 (tabela e figuras): "Os resultados das 5 configurações nos dois modelos, mais uma **busca aleatória com o mesmo orçamento** de avaliações, como base de comparação. No gráfico de convergência, o eixo x é o custo real, em modelos treinados."

3.3: "O modelo escolhido é a Regressão Logística do E1. A escolha é feita por uma validação cruzada repetida 5×3, **nunca pelo teste**. No teste, o recall foi de 95,2% para 97,6%, e os falsos negativos de 2 para 1, com um falso positivo a mais. Sendo honestos: o ganho é modesto, porque o modelo original já tinha AUC de 0,995 e cada caso maligno vale 2,4 pontos de recall. O AG ganhou principalmente baixando o limiar para 0,26."

3.4: "E o modelo em produção prevendo uma paciente: probabilidade acima do limiar, gera alerta."

4. LLM (Luiz) [3min30]

4.1: "A LLM é o GPT-OSS 120B, via Groq. A interface é única e também aceita Ollama, OpenAI e Anthropic."

4.2 (contexto): "Esta é a parte mais importante: a LLM **não recebe a paciente crua**. Ela recebe a probabilidade, o limiar, a faixa de risco e os 5 fatores que mais pesaram para *esta* paciente, calculados por oclusão, com as médias de benignos e malignos. Esses são os únicos números que ela pode usar. É o que chamamos de *grounding*."

4.3 (prompts): "Testamos três estratégias de prompt. O básico só faz a pergunta. O estruturado tem persona, regras e saída em JSON. O few-shot acrescenta um exemplo. Aqui está o prompt de sistema, com as regras: usar só os números do contexto, nunca dar diagnóstico definitivo e sempre terminar com o aviso."

4.4 (resposta, as duas células): "A chamada real: provedor, modelo, latência e tokens. Primeiro a resposta **bruta**, como veio da API, em JSON. Depois o JSON validado vira o texto que o médico lê."

4.5 (avaliação): "Cada resposta é avaliada automaticamente em formato, fidelidade numérica (se há números inventados), cobertura dos fatores, coerência com o modelo, segurança e legibilidade. E uma segunda chamada, o LLM juiz, dá notas de 1 a 5."

4.6 (básico × few-shot): "Por que o prompt importa: na mesma paciente, o básico não gera JSON, omite o aviso e escreve três vezes mais. O few-shot fica completo e mais curto."

4.7: "O médico também pode fazer uma pergunta livre sobre o caso."

4.8 (fallback): "Se a LLM falhar, aqui simulando um limite de taxa, o sistema responde com um template, mas **deixa isso explícito**: `fallback=True`, com o motivo. A resposta nunca finge ter vindo da LLM."

4.9 (relatório clínico): "A LLM também traduz a comparação original × otimizado para a equipe clínica." *(Se a LLM errar alguma conta, como já aconteceu, comentem: o critério de fidelidade confere se o número existe no contexto, não se a conta está certa. Por isso o relatório técnico traz uma conferência calculada dos dados. É uma limitação real e vale mencionar.)*

4.10 (avaliação salva): "A avaliação completa: 10 pacientes, 3 estratégias e o juiz, com **zero falhas**. Os prompts estruturados chegam a 0,98–0,99 nos critérios automáticos, contra 0,75 do básico."

5 e 6. API, monitoramento e testes (Gabriel) [1min15]

5 (chamadas): "Tudo isso é servido por uma API FastAPI, que aqui roda dentro do notebook. Vemos o `/health`, o modelo em produção, a previsão, a interpretação pela LLM, a lista de pacientes com os erros do modelo e um job do AG rodando no servidor, acompanhado até concluir."

6.1 e 6.2: "Monitoramento: métricas Prometheus por rota, previsões, chamadas à LLM com status (ok ou fallback) e a fitness do AG. Os logs saem em JSON, com o `request_id` de cada requisição."

6.3: "Escalabilidade automática: no Kubernetes, um HPA de 2 a 10 réplicas por CPU e memória, e o AG rodando como Job em lote. Localmente, `docker compose` com réplicas atrás do nginx."

6.4: "E os 75 testes automatizados passando."

Aplicação web, tudo integrado [3min30]

Trocar para o navegador em http://localhost:4201. Seletor no topo: **groq**.

Visão geral (Gabriel, 15s): "A mesma API agora numa interface, no mesmo estilo visual da Fase 1. O aviso de apoio à decisão aparece em todas as telas."

Algoritmo genético (Gustavo, 1 min): "A tabela e a convergência que vimos no notebook. Em **Rodar o AG ao vivo**, com população 12 e 8 gerações, a barra de progresso avança e o gráfico cresce geração a geração, com melhor, média e diversidade."

Paciente + LLM (Luiz, 1 min): "A jornada do médico. Filtro **Erros do modelo**: escolho a #73, um falso negativo. O modelo deu 7,8% e não alertou. Clico em **Explicar** e a LLM responde, com o selo do provedor e do modelo. Depois faço uma **Pergunta livre**." *(Esse é o caso mais interessante para discutir: o modelo errou, e a explicação mostra por quê.)*

Inspetor da LLM (Luiz, 45s): "Para conferência, tudo o que entra e sai da LLM: prompt de sistema, prompt do usuário, contexto, resposta bruta, notas automáticas, juiz, latência e tokens. Em **Comparar as 3 estratégias**, vemos lado a lado na mesma paciente."

Avaliação da LLM e Relatório (Gabriel, 30s): "A avaliação salva, com gráficos por critério e as notas do juiz por estratégia, e o relatório para a equipe clínica, que abre já salvo, sem gastar cota."

Encerramento (Gabriel) [20s]

> *"Resumindo: o algoritmo genético melhorou os dois modelos, principalmente ajustando o limiar, e reduziu os falsos negativos. A LLM explica cada caso usando só números verificáveis, e avaliamos a qualidade dessas explicações. As limitações: é um dataset público e pequeno, os pesos da fitness devem ser definidos com a equipe clínica, e nada disso substitui validação clínica. Obrigado!"*

Se o tempo apertar

Cortem nesta ordem:

1. Seção **2.2** (operadores): basta mencioná-los.
2. Seção **3.4**.
3. Seção **4.7** (a pergunta livre aparece de novo no app).
4. Seção **6.3** (mostrem só a frase sobre HPA).
5. No app, o **Inspetor** (já foi visto no notebook).

Comandos

```
cd desafio_tech_2                                # raiz do projeto
.venv\Scripts\activate                           # Windows
jupyter notebook notebooks/02_demo_fase2.ipynb   # ou abrir no VS Code
uvicorn src.api.main:app --port 8000             # API (outro terminal)
cd frontend && npm start                         # app em http://localhost:4201 (outro terminal)
```